

Course Outline: Prediction Models

Title: Prediction Models: What They Are and How They Work **Skill:** Skill x: Predictions Model

Learnpack: + Modelos de predicción **File:** s__-w16d48-modelos-de-prediccion **Prerequisite:** Skill 47

(queue-based async processing) **Target Audience:** Beginner developers entering the machine learning section of the curriculum — no prior ML knowledge assumed **Total Lessons:** 7 **Format:** Conceptual (0% hands-on)

Course Description

A prediction model is software that uses patterns in historical data to predict something about new data. Before training, selecting, or debugging models, you need to understand what they do and how the two fundamental types work: classifiers (predict a category) and regressors (predict a number). This course builds that conceptual foundation — the vocabulary and mental model you need for everything that follows.

Course Philosophy

This course emphasizes:

- Intuition before mathematics — understand what a model does before how it does it
 - Concrete examples from real applications students will recognize
 - The fundamental distinction between classification and regression as the starting point for all ML model decisions
-

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - What Is a Prediction Model	2
02 - Types of Prediction Models	2
03 - Assessment	1
04 - Outro	1
Total	7

00.0 Prediction Models

Learning Objectives:

- Understand why prediction models are valuable in software applications

- Recognize prediction models as pattern-learned functions, not explicit rules
- Preview the course structure

Content Outline:

- The core idea: a prediction model is a function that learned from data to map inputs to outputs
- The contrast with traditional programming: traditional code uses explicit rules ("if age > 18, classify as adult"); prediction models learn rules from examples ("show me 10,000 users with their ages and behaviors, and I'll learn to predict behavior")
- Why this matters for software developers: ML models are increasingly embedded in backend systems — recommenders, fraud detectors, spam filters, churn predictors — and developers who integrate, deploy, or build these systems need to understand what they're working with
- The two fundamental questions a prediction model can answer:
 1. "Which category does this belong to?" → classifier
 2. "What number should this be?" → regressor
- Course structure: Section 01 defines prediction models and their common applications; Section 02 covers the two types (classifiers and regressors); assessment tests the distinction

Transitions: This lesson frames the problem. Section 01 defines what a prediction model is.

Key Principles:

- A prediction model is not a hand-written rule set — it discovers rules from data
- "Prediction" means the model outputs a value for inputs it has never seen before
- The same architecture can be used for very different tasks — the difference is in the output type and training data

Examples:

- Spam filter: trained on millions of emails → predicts "spam" or "not spam" for new emails
- House price predictor: trained on thousands of homes → predicts the price for a new home
- Recommendation engine: trained on millions of user-item interactions → predicts what a specific user will like

01.0 What Is a Prediction Model

Learning Objectives:

- Define a prediction model in precise terms: a function that maps inputs to outputs by learning from historical data
- Distinguish a prediction model from hard-coded business logic
- Identify the three components every prediction model requires: training data, a learning algorithm, and a prediction function

Content Outline:

- Precise definition: a prediction model is a mathematical function $f(x) \rightarrow y$ where:
 - x is the input (features describing the new item to predict)
 - y is the output (the predicted value or category)

- f was learned from a training dataset where both x and y were known
- What "training" means: the learning algorithm adjusts the model's internal parameters to minimize prediction error on the training data
- What "prediction" means: for a new input x (where the true y is unknown), the trained model computes $f(x)$ as its best estimate of y
- The three components:
 1. **Training data:** historical examples with known inputs and outcomes — the model can only predict what it has seen examples of
 2. **Learning algorithm:** the process that fits the model's parameters to the training data (e.g., gradient descent, decision tree splitting)
 3. **Prediction function:** the trained model — takes new inputs and produces predictions
- Why models predict rather than know: the model extrapolates patterns from training data to new data; if the new data is very different from training data, predictions are unreliable
- Section preview: 01.1 shows the wide range of real applications where this pattern appears

Transitions: You understand what a model is. 01.1 shows where you'll encounter them.

Key Principles:

- A model's predictions are only as good as its training data — garbage in, garbage out
- The model doesn't "know" anything; it generalizes patterns from examples
- Every prediction has uncertainty; a good model also quantifies its confidence

Examples:

- Hard-coded rule: `if credit_score > 700: approve_loan()`
- Prediction model: `model.predict(applicant_features) → "approved" or "denied"` (learned from 500,000 past loan decisions and outcomes)
- The model learns more nuanced patterns: not just credit score, but age, income, loan amount, and hundreds of other features interacting in complex ways

01.1 Where Prediction Models Are Used

Learning Objectives:

- Recognize prediction models across diverse real-world applications
- Map each application to its prediction task (what is being predicted)
- Understand why software developers — not just data scientists — need to understand prediction models

Content Outline:

- Why this matters for developers: prediction models are increasingly deployed as services within backend systems; understanding what they predict helps developers integrate them correctly
- **Applications by domain:**
 - **E-commerce and recommendations:** Amazon predicts which product a user will buy; Netflix predicts which show a user will watch; Spotify predicts which song to play next — each is a prediction model outputting probabilities or scores

- **Finance and fraud detection:** banks predict whether a transaction is fraudulent; Stripe predicts the probability a business is a fraud risk; credit card companies predict whether a cardholder will default
- **Content and spam filtering:** Gmail predicts whether an email is spam; social networks predict whether a post violates policies; search engines predict which results are most relevant to a query
- **Operations and maintenance:** manufacturers predict when a machine will fail (predictive maintenance); logistics companies predict delivery delays; cloud providers predict when a server will need maintenance
- **Healthcare:** hospitals predict patient readmission risk; diagnostic tools predict disease probability from medical images; drug companies predict which compounds will bind to a target protein
- The developer's role with prediction models:
 - Integrating a model's API into a backend service
 - Preprocessing input data to match what the model expects
 - Handling model outputs (probabilities, scores, labels) in application logic
 - Monitoring model performance after deployment (detecting drift)

Transitions: You now know where prediction models appear. Section 02 covers the two fundamental types.

Key Principles:

- Prediction models appear in virtually every industry — recognizing them is the first step to working with them effectively
- The output format (a category, a number, a probability, a score) determines how the developer uses the prediction in application logic
- Most prediction models in production are not built by the developer integrating them — understanding the interface is more important than understanding the training process

Examples:

- Developer integration: `POST /api/v1/predict {"email_body": "..."} → {"prediction": "spam", "confidence": 0.97}` — the developer handles the 0.97 confidence threshold
- Real-time fraud detection: every payment triggers a model prediction in < 100ms — the developer builds the integration, not the model
- Feature preprocessing: the model expects `normalized_age` (0-1 range), not raw age — the developer writes the preprocessing code

02.0 Types of Prediction Models

Learning Objectives:

- Name the two fundamental types of prediction models: classifiers and regressors
- Explain the key distinction: classifiers predict a category; regressors predict a number
- Identify which type is appropriate for a given prediction task

Content Outline:

- The single most important distinction in ML: what type of output does the model produce?
 - **Classifier:** predicts a discrete category (label) — one of a finite set of classes
 - **Regressor:** predicts a continuous number — any value in a range
- Why this distinction matters:
 - Different evaluation metrics: classifiers use accuracy, precision, recall; regressors use MAE, RMSE
 - Different training objectives: classifiers minimize classification error; regressors minimize prediction error on a numeric scale
 - Different output handling: classifiers output a class label (or probability per class); regressors output a number
- The question to ask: "Is the output I need a category or a number?"
 - Categories: spam/not spam, approved/denied, dog/cat/bird, positive/negative sentiment
 - Numbers: house price, temperature tomorrow, time to churn, conversion probability
- Important edge case: when a regressor outputs a probability (0 to 1), developers sometimes treat values above 0.5 as one class and below as another — this is applying a threshold to turn a regressor into a binary classifier
- Section preview: 02.1 goes deeper into how each type works and provides more examples

Transitions: You know the two types exist. 02.1 explains how each works and when to use each.

Key Principles:

- Classifiers output categories; regressors output numbers — this single distinction drives every other design decision
- A "binary classifier" (two classes) and a "multi-class classifier" (many classes) are both classifiers — the distinction is in the output type, not the number of classes
- Some tasks are naturally one type: predicting "will this customer churn?" is classification; predicting "how many days until this customer churns?" is regression

Examples:

- Classifier output: `predict("is this email spam?")` → "spam" or "not spam"
- Regressor output: `predict("what is the expected delivery time?")` → 3.7 days
- Edge case: `predict("probability of fraud")` → 0.83 (a number from a regressor, thresholded at 0.5 to become "fraud" or "not fraud")

02.1 Classifiers vs Regressors

Learning Objectives:

- Explain how a classifier works conceptually: finds decision boundaries in feature space
- Explain how a regressor works conceptually: fits a function to continuous output values
- Apply the classifier/regressor distinction to new scenarios with confidence

Content Outline:

- **How a classifier works:**
 - A classifier draws boundaries in feature space to separate different classes

- Conceptual example: spam classifier with two features (word count, presence of "buy now")
 - Training: sees thousands of spam/not spam emails with these features
 - Learning: finds that emails with high word count AND "buy now" → spam; others → not spam
 - Prediction: for a new email, places it in feature space and returns the class of that region
- The boundary can be a line (linear classifier), a curve, a tree structure, or a complex neural network boundary — what matters is it separates classes
- Output: a class label (and optionally a probability per class: "87% spam, 13% not spam")
- Common classifier types: logistic regression, decision trees, random forests, support vector machines, neural networks
- Binary classifier: two classes (spam/not spam, fraud/not fraud, approved/denied)
- Multi-class classifier: many classes (cat/dog/bird, sentiment: positive/neutral/negative, language: English/Spanish/French)
- **How a regressor works:**
 - A regressor fits a function to predict a continuous output
 - Conceptual example: house price prediction with features (size, location, age)
 - Training: sees thousands of homes with these features AND their actual sale prices
 - Learning: fits a function that maps (size, location, age) → price
 - Prediction: for a new home, plugs features into the function → predicted price
 - The function can be a line (linear regression), a curve, a tree structure, or a complex neural network — what matters is it outputs a number
 - Output: a number (price, temperature, duration, probability)
 - Common regressor types: linear regression, decision trees (regression variant), random forests (regression variant), neural networks

- **Choosing between them:**

Prediction Task	Type
Is this email spam?	Classifier
What is tomorrow's temperature?	Regressor
Will this customer churn (yes/no)?	Classifier
How many days until this customer churns?	Regressor
Which product will this user buy?	Classifier
How much will this user spend?	Regressor

- **The probability bridge:** many classifiers output a probability rather than just a class — $P(\text{spam}) = 0.87$. The developer applies a threshold to convert: `if  $P(\text{spam}) > 0.5$  → "spam"`. This lets you tune the threshold based on business needs (e.g., for fraud detection, use 0.3 to catch more fraud at the cost of more false positives).

Transitions: You can now identify and distinguish classifiers and regressors. The assessment tests your ability to apply this distinction.

Key Principles:

- Both types are prediction models that learn from data — the difference is only in the output type
- Algorithms like decision trees and neural networks can implement BOTH classifiers and regressors — the algorithm choice is less fundamental than the task definition
- Most ML problems can be framed as either classification or regression; choosing correctly is the first design decision

Examples:

- "Will it rain tomorrow?" → Classifier (yes/no) — but "How much rain tomorrow?" → Regressor (millimeters)
 - "What score will a student get?" → Regressor (0-100) — but "Will a student pass or fail?" → Classifier
 - "What is the risk of this loan defaulting?" → Could be classifier (default/no default) OR regressor (probability 0-1) depending on how you want to use the output
-

03.0 Assessment 🧐

Learning Objectives:

- Correctly classify each prediction task as requiring a classifier or regressor
- Apply the conceptual understanding of how each model type works to new scenarios
- Recognize real-world applications of prediction models

Question Format: Scenario-based (multiple-choice)

Questions:

1. A company wants to predict whether a job applicant will succeed in the role. The output is "likely to succeed" or "unlikely to succeed". What type of model is this?
 - A) Regressor
 - B) Classifier ✓
 - C) It depends on the algorithm
 - D) Neither — this isn't a prediction model
2. A streaming service wants to predict how many minutes a user will watch today. What type of model is needed?
 - A) Classifier
 - B) Regressor ✓
 - C) Both — a hybrid model
 - D) No model needed — this is a simple calculation
3. What is the key difference between a classifier and a regressor?
 - A) Classifiers require more training data than regressors
 - B) Classifiers predict categories; regressors predict numbers ✓
 - C) Regressors are always more accurate
 - D) Classifiers can only work with text data

4. A model returns the output `{"prediction": "fraud", "confidence": 0.94}` for a credit card transaction. What type of model is this?
- A) Regressor (because 0.94 is a number)
 - B) Classifier ✓
 - C) Neither — fraud detection doesn't use ML
 - D) A regressor that was converted to a classifier using a threshold
5. The "training data" for a prediction model refers to:
- A) The rules explicitly programmed by the developer
 - B) Historical examples with known inputs and outputs used to fit the model ✓
 - C) The model's internal code
 - D) The documentation for the model's API
6. Which of the following is best framed as a REGRESSOR task?
- A) Classifying whether a document is positive or negative sentiment
 - B) Predicting which of five categories a customer will choose
 - C) Predicting the electricity consumption of a building next month (in kWh) ✓
 - D) Detecting whether an image contains a cat or a dog
7. A developer integrates a spam filter into an email service. The model returns a probability between 0 and 1. The developer marks emails as spam if the probability is above 0.5. What is the developer applying?
- A) A training algorithm
 - B) A feature engineering step
 - C) A threshold to convert a probability into a binary classification decision ✓
 - D) A data preprocessing step
8. Why can the same algorithm (e.g., a decision tree) be used for both classification and regression?
- A) Because decision trees always output probabilities, which work for both
 - B) The algorithm is the same; the output type and training objective differ depending on the task ✓
 - C) Decision trees can only be used for classification
 - D) Regression trees and classification trees are completely different algorithms

Evaluation Criteria:

- Correctly distinguishes classifiers from regressors based on output type
- Understands training data's role
- Applies the threshold concept for probability outputs
- Recognizes the same algorithm can serve both tasks

Score Interpretation:

- 8/8: You're ready for the hands-on prediction model learnpacks
- 6–7: Review any missed lessons before moving on

- 4–5: Re-read Section 02 with focus on the type distinction
 - Below 4: Restart from Section 01
-

04.0 Models in Your Stack

Content Outline:

- Course recap: what students accomplished
 - Defined a prediction model: a function $f(x) \rightarrow y$ learned from training data
 - Understood what "training" and "prediction" mean at a conceptual level
 - Recognized prediction models across diverse real-world applications
 - Mastered the fundamental distinction: classifiers predict categories, regressors predict numbers
 - Learned that the same algorithm can implement either type
- Connection to bigger picture: machine learning models are increasingly a standard component in modern software stacks. Understanding what they predict, how to use their outputs, and what their limitations are is becoming a baseline skill for software developers — not just data scientists.
- Next steps:
 - The next learnpack covers where to find pre-trained models and how to load and run them — no training required
 - Subsequent learnpacks cover how to train classifiers and regressors from scratch
 - Later learnpacks cover debugging, orchestration, and monitoring
- Resources for further exploration:
 - Google's Machine Learning Crash Course — the conceptual foundation, explained clearly
 - scikit-learn documentation's "choosing the right estimator" guide — practical decision tree for model selection
 - Kaggle — datasets and competitions where classifiers and regressors are built and evaluated in context
- Encouragement: prediction models are not magic — they are functions that learned from data. Every time you've interacted with a recommendation, a spam filter, or an estimated delivery time, you've been on the receiving end of this simple idea. Understanding what these systems do is the first step to building them, integrating them, and improving them.

Note: This is conclusion only — no theory, exercises, or assessment.